1 - Containerization and application layer load balancing

Name of report: `LS_LAB_1_Nikita_Niakhai`

Course: DevOps and Security

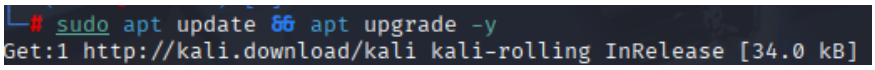
Performed by *Nikita Niakhai 18*

Date submission: 27.01.2026

Task 1: Get familiar with Docker Engine

1. Pull Nginx v1.23.3 image from dockerhub registry and confirm it is listed in local images

- I updated packages



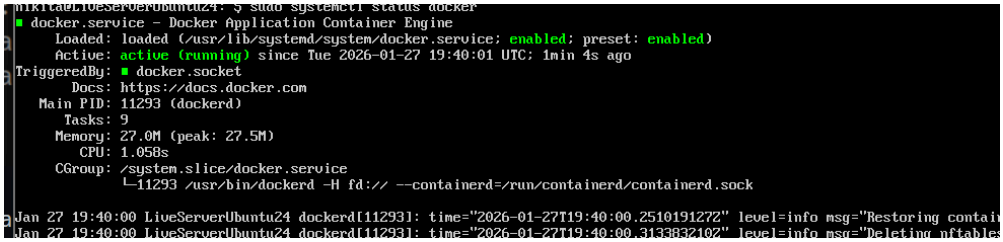
- I Set up Docker's `apt` repository [1]

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
```

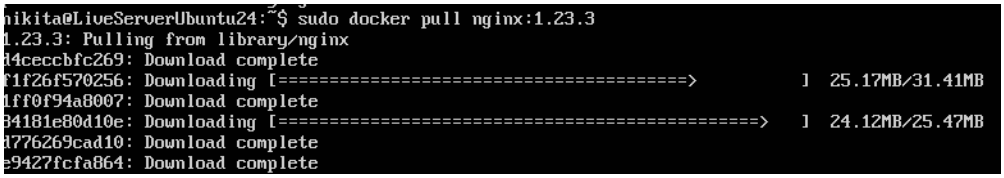
```
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

- I installed latest docker

```
sudo apt install docker-ce docker-ce-cli containerd.io docker
-buildx-plugin docker-compose-plugin
```



- Pulled nginx



2. Run the pulled Nginx as a container with the below properties
   a. Map the port to 8080.
   b. Name the container as `nginx-st18`
   c. Run it as daemon .

```
root@LiveServerUbuntu24:~# docker run -d --name nginx-st18 -p 8080:80 nginx:1.23.3
5c1f857fd019858d8daaa381cd3047a8e5a30f7945a8d033ef17f19e830f2303
```

3. Confirm port mapping.

- List open ports in host machine. Verified running status in proccesses:

```
root@LiveServerUbuntu24:~# netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp6       0      0 :::8080                 :::*                    LISTEN
udp        0      0 127.0.0.54:53           0.0.0.0:*
udp        0      0 127.0.0.53:53           0.0.0.0:*
udp        0      0 10.0.2.15:68            0.0.0.0:*
```

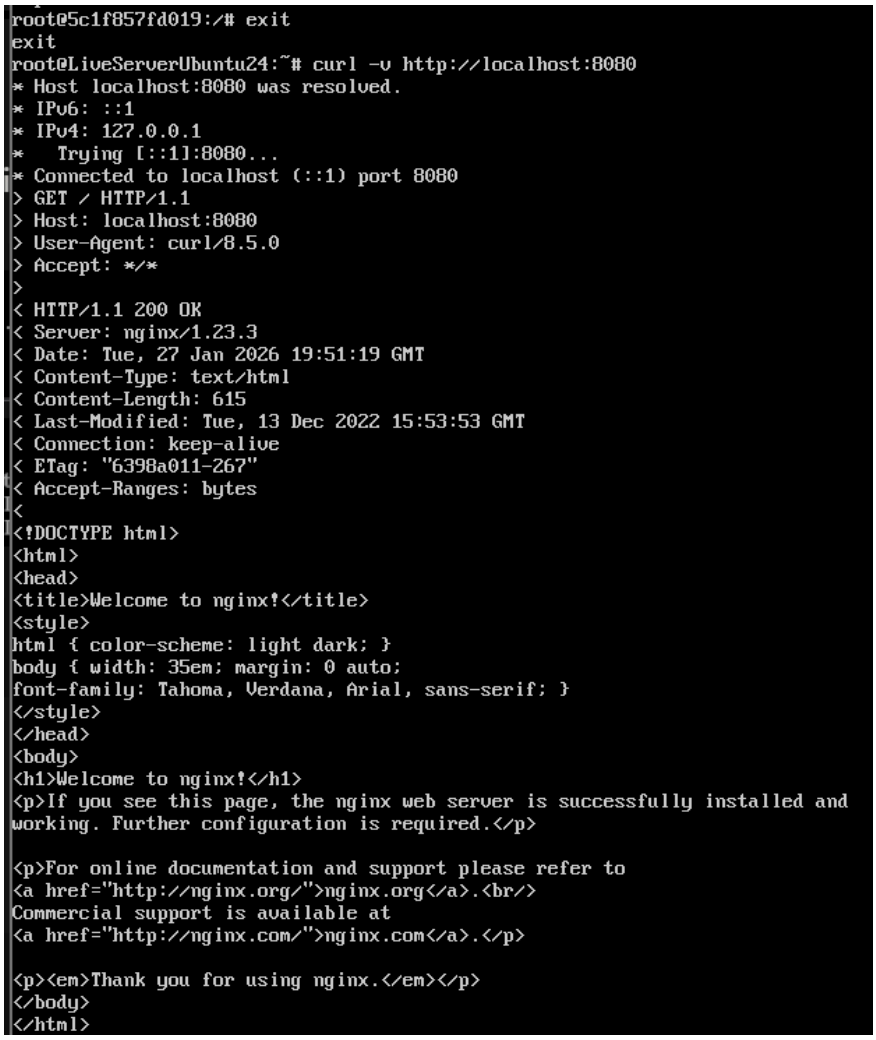
- I accesses docker and updated packages there and installed `net-tools`. List open ports inside the running container.

```
root@5c1f857fd019:/# apt update && apt upgrade
Get:1 http://deb.debian.org/debian bullseye InRelease [75.1 kB]
Get:2 http://deb.debian.org/debian-security bullseye-security InRelease [27.2 kB]
Get:3 http://deb.debian.org/debian bullseye-updates InRelease [44.0 kB]
Get:4 http://deb.debian.org/debian bullseye/main amd64 Packages [8066 kB]
35% [4 Packages 2208 kB/8066 kB 27%]
```

```
root@5c1f857fd019:/# netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp6       0      0 :::80                   :::*                    LISTEN
```

- I exited from docker CLI and accessed the page. And saw default nginx page

```
# With verbose output (shows headers)
curl -v http://localhost:8080
```



4. Create a Dockerfile similar to the below properties (let’s call it container A).

- Image tag should be Nginx v1.23.3.
- Create a custom index.html file and copy it to your docker image to replace the Nginx default web
  page.
- Build the image from the Dockerfile, tag it during build as `nginx:st18`, check/validate local images, and
  run your custom made docker image.
- Stopped prev container to be able to access port 8080 and runed containerA

```
root@LiveServerUbuntu24:~# echo '<h1>st18</h1>' > index.html
root@LiveServerUbuntu24:~# echo -e 'FROM nginx:1.23.3\n\n' > Dockerfile
root@LiveServerUbuntu24:~# echo -e 'COPY index.html /usr/share/nginx/html/index.html\n\n' >> Dockerfile
```

```
root@LiveServerUbuntu24:~# docker build -t nginx:st18 .
```

```
root@LiveServerUbuntu24:~# docker run -d --name containerA -p8080:80 nginx:st18
bf8821cdfd66ba8db62260cfa0439d995dc36df7d39831801cc363023e134787
```

- Access via browser and validate that your custom page is hosted.

```
root@LiveServerUbuntu24:~# curl http://localhost:8080
<h1>st18</h1>
```

Task 2: Work with multi-container environment

1. Create another Dockerfile similar to step 1.4 (Let's call it container B), and an index.html with different content.

```
root@LiveServerUbuntu24:~# tree
.
├── containerA
│   ├── Dockerfile
│   └── index.html
└── containerB
    ├── Dockerfile
    └── index.html
```

```
root@LiveServerUbuntu24:~# cat ~/containerB/index.html
<h1>Hi bro, I am from different container B image! ST18</h1>
root@LiveServerUbuntu24:~# cat ~/containerB/Dockerfile
FROM nginx:1.23.3


COPY index.html /usr/share/nginx/html/index.html
```

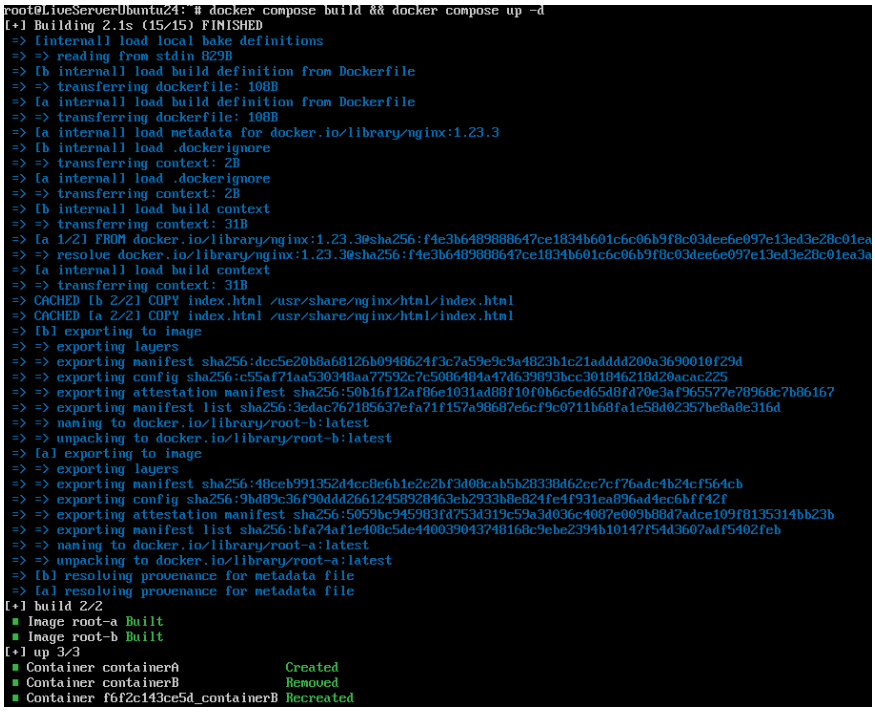
- Created

2. Write a docker-compose file with the below properties (Multi-build: Builds both Dockerfiles and runs both images; Port mapping: Container A should listen to port 8080 and container B should listen to port 9090. (They host two different web pages)

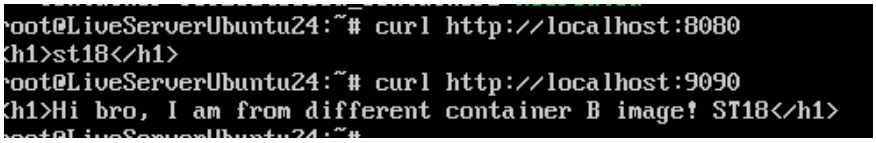
- I wrote it

```
 GNU nano 7.2                                    docker-compose.yaml *
services:
  a:
    container_name: containerA
    build: containerA
    ports: [ "8080:80" ]
  b:
    container_name: containerB
    build: containerB
    ports: [ "9090:80" ]
```

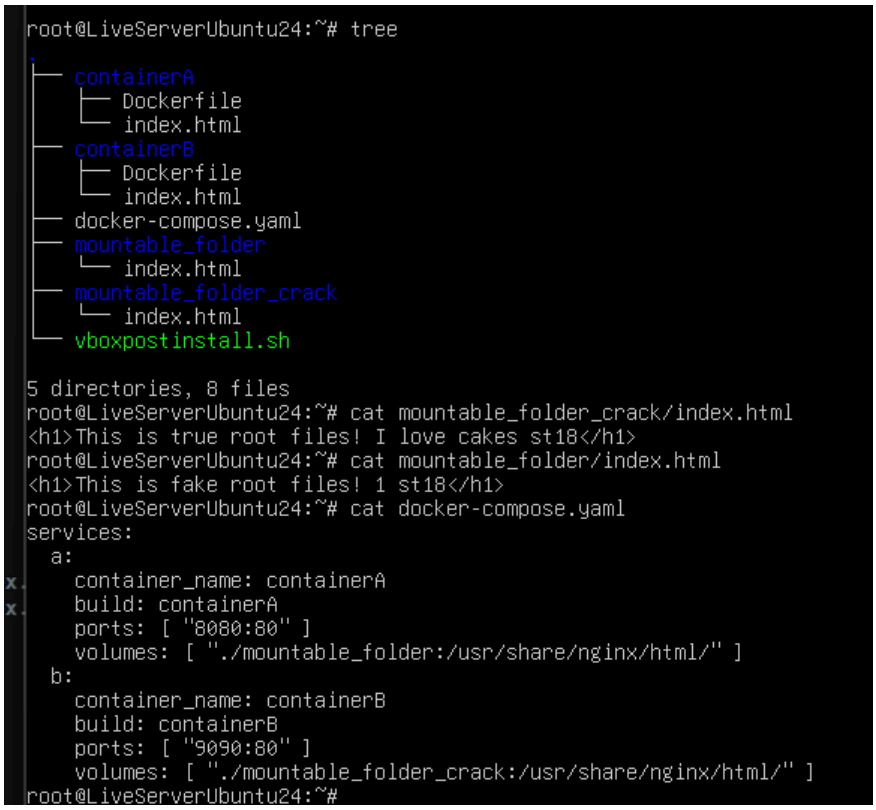
- I run docker images

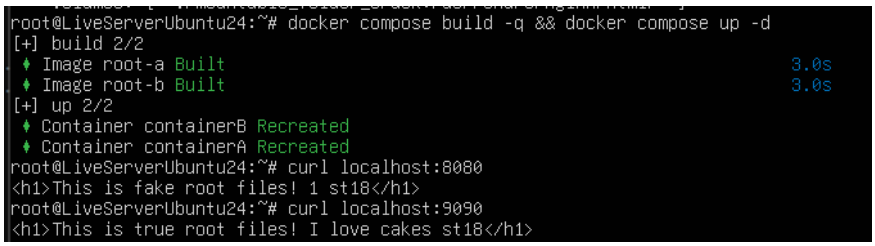


- I confirm both websites are accessible



- Volumes: Mount (bind) a directory from the host file system to Nginx containers and update the
  contents of index.html in the host file system, re-deploy and confirm in the browser that the web
  page's content is updated.





3. Configure L7 Loadbalaner

- Install Nginx in the host machine or add a third container in the docker-compose that will act as
  loadbalancer, and configure it in front of two containers in a manner that it should distribute the load in a Weighted Round Robin approach.
- Added configuration for docker-compose

```
 GNU nano 7.2                                                  docker-compose.yaml
ervices:
 a:
   container_name: containerA
   build: containerA
   ports: [ "8080:80" ]
   volumes: [ "./mountable_folder:/usr/share/nginx/html/" ]
 b:
   container_name: containerB
   build: containerB
   ports: [ "9090:80" ]
   volumes: [ "./mountable_folder_crack:/usr/share/nginx/html/" ]
 lb:
   container_name: lb
   image: nginx:1.23.3
   ports: [ "80:80" ]
   depends_on: [ "a", "b" ]
   volumes: [ "./nginx.conf:/etc/nginx/nginx.conf" ]_
```

- Added nginx for load balancing

```
  GNU nano 7.2                                                        nginx.conf
worker_processes auto;

events {
        worker_connections 1024;
}

http {
        upstream backend {
                server a:80 weight=3;
                server b:80 weight=1;
        }

        server {
                listen 80;
                location / {
                        proxy_pass http://backend;
                }

        }
}
```

- Entered lb container, updated packages and installed nginx
- Updated container and created new

```
root@LiveServerUbuntu24:~# tree
.
├── containerA
│   ├── Dockerfile
│   └── index.html
├── containerB
│   ├── Dockerfile
│   └── index.html
├── docker-compose.yaml
├── mountable_folder
│   └── index.html
├── mountable_folder_crack
│   └── index.html
├── nginx.conf
└── vboxpostinstall.sh

5 directories, 9 files
root@LiveServerUbuntu24:~# docker compose build -q && docker compose up -d
[+] build 2/2
 ✔ Image root-a Built                                                    2.7s
 ✔ Image root-b Built                                                    2.7s
[+] up 3/3
 ✔ Container containerA Recreated
 ✔ Container containerB Recreated
 ✔ Container lb         Created
```

- Made 10 curls and saw that weights for load balancing divided correctly. Accessed the page of Nginx ALB and validated, it is load-balancing the traffic.